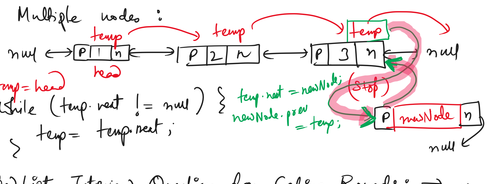
Insert Operation in a DLL :→

① head is null (list is empty)
create node & make the new node the head of list.

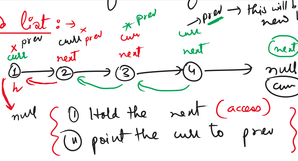
② Multiple nodes :



Linked List Interview Questions for Coding Rounds :→

* **Reverse a linked list :→**

prev → this will become the new head!




```
Node prev = null;
Node curr = head;
Node next = null;
while (curr != null) {
    next = curr.next;
    curr.next = prev;
    prev = curr;
    curr = next;
}
{ head = prev; }
```

null { ① Hold the next (access) ② point the curr to prev }

④ → ③ → ② → ① → null

List ① → ② → ③ → ④ → ⑤ → ⑥ → null

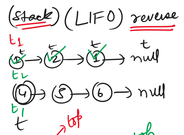
```
static void PrintEven ( ) {
    2, 4, 6
}
static void printOdd ( ) {
    1, 3, 5
}
```

Important Placement Drive Coding Questions on Linked Lists :→
{ Leet Code, Coding Ninjas, GFG }

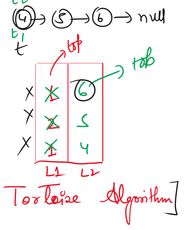
1. Reverse a linked list
2. Reverse a linked list in groups of k
3. Rotate a linked list
4. Floyd's Algorithm or Hare & Tortoise Algorithm
   a) Middle of a linked list / Mid Point of Linked List
   b) Delete Middle of a linked list
   c) Cycle Detection in a linked list
5. Merge two sorted linked lists
6. Intersection Point of two linked lists.
7. Add two numbers using linked lists.
8. Palindrome Linked List.

Palindrome Linked List :→

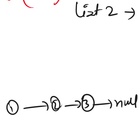
(stack) (LIFO) reverse




```
boolean isPalindromeList(head)
{
    true;
    false;
}
```



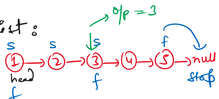
Reverse a LL :



Floyd's Algorithm or [Hare & Tortoise Algorithm]

* Fast & Slow Points
2 steps 1 step

* Find the middle of a linked list :

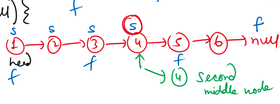


Pseudo code :

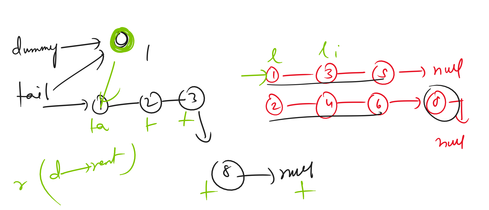
```
Node slow = head;
Node fast = head;
while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
return slow.data;
```

TC1

TC2



Floyd's Cycle Detection Algo :→

Slow & Fast Pointers:
1 2

```
Node slow = head;
Node fast = head;
while (fast != null && fast.next != null)
{
    slow = slow.next;
    fast = fast.next.next;
    if (slow == fast) return true;
}
return false;
```